AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE
FARIDABAD – 121002



Mini High-Frequency Trading Simulation using High Performance Computing

MID-SEMESTER PROJECT REPORT

Submitted in partial fulfilment of the requirements
for the course

HIGH PERFORMANCE COMPUTING

Submitted by

P. Bhargav Ram	DL.AI.U4AID24146
Vishnu Karthik C	DL.AI.U4AID24109
Pujya Ganesh Karthik	DL.AI.U4AID24141
Amith Vignesh K	DL.AI.U4AID24116

Under the guidance of

Suraj Singh
Assistant Professor

B.Tech – Artificial Intelligence and Data Science
2026



BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"Mini High-Frequency Trading Simulation using High Performance Computing"** is the bonafide work of

P. Bhargav Ram (DL.AI.U4AID24146)
Vishnu Karthik C (DL.AI.U4AID24109)
Pujya Ganesh Karthik (DL.AI.U4AID24141)
Amith Vignesh K (DL.AI.U4AID24116)

who carried out the project work under the guidance and supervision of **Suraj Singh**, Assistant Professor, Amrita School of Artificial Intelligence, Faridabad.

Project Guide	**Head of Department**
Suraj Singh	**Dr. Vijay Kumar Soni**
Assistant Professor	Head of Department
School of AI, Faridabad	School of AI, Faridabad



DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled **"Mini High-Frequency Trading Simulation using High Performance Computing"** submitted as part of the Bachelor of Technology programme is a record of the project work carried out by us under the guidance of **Suraj Singh**.

The work presented in this report has been prepared for academic purposes and has not been submitted elsewhere for the award of any other degree or diploma.

Signatures of the Project Team:

_____ P. Bhargav Ram
(DL.AI.U4AID24146)

_____ Vishnu Karthik C
(DL.AI.U4AID24109)

_____ Pujya Ganesh Karthik
(DL.AI.U4AID24141)

_____ Amith Vignesh K
(DL.AI.U4AID24116)

ACKNOWLEDGEMENT

We express our sincere gratitude to **Sri Mata Amritanandamayi Devi**, Chancellor of Amrita Vishwa Vidyapeetham, for her blessings and inspiration.

We are grateful to the faculty members of the Amrita School of Artificial Intelligence for providing the facilities and academic environment required to complete this project.

We would especially like to express our sincere gratitude to our project guide, **Suraj Singh**, for his continuous guidance, valuable suggestions and encouragement throughout the development of this work.

We also thank the review panel members and the faculty of the Department of Artificial Intelligence and Data Science for their constructive feedback and suggestions.

Project Team

- P. Bhargav Ram (DL.AI.U4AID24146)
- Vishnu Karthik C (DL.AI.U4AID24109)
- Pujya Ganesh Karthik (DL.AI.U4AID24141)
- Amith Vignesh K (DL.AI.U4AID24116)

ABSTRACT

This project presents a compact High-Frequency Trading (HFT) simulation implemented in the C programming language to demonstrate fundamental concepts of High Performance Computing (HPC). The system generates synthetic financial quote data, maintains the current best bid and ask values using a Level-1 order book, applies a simple momentum-based trading strategy, calculates profit and loss (P&L), and measures execution time and event-processing throughput.

The same symbol-level simulation is implemented using four computing approaches: sequential execution, POSIX Threads (Pthreads), OpenMP and the Message Passing Interface (MPI). Since financial symbols can be processed independently in the current design, the workload can be distributed among multiple threads or processes.

The main purpose of the project is to compare different parallel programming techniques while preserving identical trading logic. The project is intentionally kept compact for the mid-semester stage and focuses on parallel-computing concepts rather than reproducing a complete financial exchange or developing a profitable trading system.

Keywords: High Performance Computing, High-Frequency Trading, C, Pthreads, OpenMP, MPI, Parallel Computing, Level-1 Order Book.

TABLE OF CONTENTS

Contents

1 Introduction **1**

1.1 Problem Statement 1

1.2 Objectives 1

2 Literature Review **2**

2.1 Sequential Processing 2

2.2 POSIX Threads 2

2.3 OpenMP 3

2.4 Message Passing Interface 3

2.5 HFT Simulation and Level-1 Order Book 3

2.6 Identified Gap 4

3 Proposed Methodology **4**

3.1 Stage 1: Synthetic Market Data Generation 4

3.2 Stage 2: Market Data Loading and Symbol Grouping 5

3.3 Stage 3: Level-1 Order Book Update 5

3.4 Stage 4: Momentum Trading Strategy 6

3.5 Stage 5: Profit and Loss Calculation 6

3.6 Stage 6: Parallel Execution 7

3.6.1 Sequential Version 7

3.6.2 Pthreads Version 7

3.6.3 OpenMP Version 7

3.6.4 MPI Version 8

4 System Workflow **8**

5 Implementation **9**

6 Performance Metrics **9**

6.1 Execution Time 9

6.2 Throughput 9

6.3 Speedup 10

7 Validation and Current Results **10**

8 Limitations **11**

9 Conclusion **12**

10 Future Scope **12**

1. Introduction

High-Frequency Trading systems process financial-market information and generate trading decisions within short time intervals. In practical electronic markets, large volumes of quotes, orders and price updates must be processed efficiently. As the amount of market data increases, the computational requirement also increases.

High Performance Computing provides techniques for dividing computational workloads among multiple CPU cores, threads or processes. Financial-market simulation is therefore a suitable application for demonstrating parallel-computing concepts.

For this mid-semester project, a complete exchange-level order-matching engine would introduce unnecessary complexity. The implemented system therefore uses a simplified architecture based on independent financial symbols, deterministic synthetic quote events and a Level-1 order book.

The overall processing pipeline is shown below.

CSV Market Data → Level-1 Order Book → Momentum Signal → Buy/Sell Decision → P&L Calculation → Performance Measurement

Each symbol is simulated independently. This creates a natural opportunity to distribute the workload using Pthreads, OpenMP and MPI.

1.1 Problem Statement

A sequential financial-market simulation processes one symbol after another. When the number of symbols or events becomes larger, total execution time may increase.

The main problem considered in this project is therefore:

Problem Statement
How can independent-symbol HFT simulation workloads be distributed using different parallel programming techniques while maintaining identical trading results?

1.2 Objectives

The main objectives of the project are:

- To generate deterministic synthetic financial-market quote data.
- To maintain the latest best bid and best ask using a simple Level-1 order book.
- To implement a basic momentum-based trading strategy.
- To calculate the number of trades, final position and P&L.
- To develop Sequential, Pthreads, OpenMP and MPI implementations.
- To compare execution time and event-processing throughput.
- To confirm that parallel execution does not change the underlying trading result.

2. Literature Review

High Performance Computing provides several programming models for executing computational workloads efficiently. The choice of a parallelization method depends on whether processors share memory, operate in separate address spaces, or communicate over a distributed system.

This project studies three widely used parallel programming approaches: POSIX Threads, OpenMP and MPI, using sequential execution as the baseline.

2.1 Sequential Processing

Sequential execution processes instructions using a single execution path. It is simple to implement and provides a useful baseline for comparing parallel versions.

In the current HFT simulator, the sequential version processes the financial symbols one after another.

Although this approach is easy to understand and debug, it cannot directly use multiple CPU cores for symbol-level processing.

2.2 POSIX Threads

POSIX Threads, commonly referred to as Pthreads, provide explicit multithreading support in C.

The programmer is responsible for creating threads, assigning work, managing thread-specific data and joining threads after execution.

In this project, symbols are distributed among available threads using a round-robin arrangement. Each thread processes its assigned symbols independently before the

results are combined.

Pthreads provide direct control over multithreading but require more manual programming than OpenMP.

2.3 OpenMP

OpenMP is a shared-memory parallel programming model that uses compiler directives to parallelize suitable regions of C/C++ programs.

The independent symbol-processing loop in this project can be parallelized using an OpenMP `parallel for` directive.

A reduction operation is used to combine local values such as trade count, final position and P&L.

OpenMP requires less explicit thread-management code than Pthreads, making it convenient for loop-level parallelism.

2.4 Message Passing Interface

The Message Passing Interface (MPI) is widely used for process-based and distributed-memory parallel computing.

Each MPI process has its own address space. Processes communicate and combine their results using MPI communication functions.

In the current project, symbols are distributed among MPI ranks. Each rank calculates local results, which are then combined using `MPI_Reduce`.

This approach can later be extended from a single machine to a multi-node system.

2.5 HFT Simulation and Level-1 Order Book

Real electronic exchanges maintain detailed limit order books containing several price levels and many active orders.

For the current mid-semester project, the implementation uses a simpler Level-1 quote book containing only:

- best bid price,
- best ask price,
- best bid quantity, and
- best ask quantity.

This simplified design reduces implementation complexity while still providing a useful market-data processing workload for comparing HPC programming approaches.

2.6 Identified Gap

Complete HFT systems may involve exchange networking, full order matching, market microstructure, complex strategies and real-time hardware.

Such systems are beyond the scope of a mid-semester HPC project.

Therefore, this project concentrates on a smaller and more manageable problem: executing the same independent-symbol simulation using different parallel programming models while preserving the same financial output.

3. Proposed Methodology

The methodology consists of six major stages:

1. Synthetic market data generation
2. Market data loading and symbol grouping
3. Level-1 order-book update
4. Momentum-based trading
5. P&L calculation
6. Parallel execution and performance measurement

3.1 Stage 1: Synthetic Market Data Generation

The program `generate_data.c` generates deterministic synthetic financial quote data and stores it in CSV format.

Each market event contains:

- simulation step,
- symbol identifier,
- bid price,
- ask price,
- bid quantity, and

- ask quantity.

The dataset size can be controlled by selecting the number of symbols and the number of events generated for each symbol.

For example:

```
./bin/generate_data 4 10000 data/market.csv
```

This command generates four symbols with 10,000 events for each symbol.

Therefore,

$$4 \times 10,000 = 40,000$$

market events are created.

3.2 Stage 2: Market Data Loading and Symbol Grouping

The generated CSV file is read by the simulator.

Events are grouped according to their symbol identifier and stored in the `SymbolData` structure.

The separation of data by symbol is important because the symbols are independent in the current simulation. Therefore, different symbols can be assigned to different worker threads or MPI processes without modifying the trading logic.

3.3 Stage 3: Level-1 Order Book Update

For each market event, the latest bid and ask information is stored.

The best bid is defined as:

$$P_{\text{bid}} = \text{Current Event Bid Price}$$

and the best ask is:

$$P_{\text{ask}} = \text{Current Event Ask Price}$$

The current mid-price is calculated using:

$$P_{\text{mid}} = \frac{P_{\text{bid}} + P_{\text{ask}}}{2}$$

The mid-price provides a simple representation of the current market price.

3.4 Stage 4: Momentum Trading Strategy

The trading strategy compares the current mid-price with the previous mid-price.

The decision rule is:

$$\text{Action} = \begin{cases} \text{Buy one unit}, & P_t > P_{t-1} \\ \text{Sell one unit}, & P_t < P_{t-1} \\ \text{No action}, & P_t = P_{t-1} \end{cases}$$

If the price increases, the simulator buys one unit at the ask price.

If the price decreases, the simulator sells one unit at the bid price.

The trading position is restricted to:

$$-10 \leq \text{Position} \leq 10$$

The strategy is intentionally simple. Its purpose is to create a repeatable processing workload for HPC experiments rather than to demonstrate real-world trading profitability.

3.5 Stage 5: Profit and Loss Calculation

The simulator maintains cash and current position throughout the simulation.

After all events for a symbol have been processed, the final mark-to-market profit and loss is calculated as:

$$\text{P\&L} = \text{Cash} + (\text{Final Position} \times \text{Last Mid-Price})$$

For every symbol, the simulation records:

- number of completed trades,
- final position,
- cash balance,
- final mid-price, and
- final P&L.

3.6 Stage 6: Parallel Execution

A major design decision in the project is that all implementations use the same `simulate_symbol()` function.

Therefore, the trading algorithm itself remains unchanged between the Sequential, Pthreads, OpenMP and MPI versions.

Only the method used to distribute symbols is different.

3.6.1 Sequential Version

The sequential version processes all symbols one after another:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \cdots \rightarrow S_n$$

Only one execution path is used.

3.6.2 Pthreads Version

The Pthreads version creates several CPU threads.

Symbols are distributed approximately using:

$$s = \text{thread_id}, \text{thread_id} + \text{thread_count}, \ldots$$

Each thread stores its own local results.

After all threads finish, the main program combines their output using `pthread_join()`.

3.6.3 OpenMP Version

The OpenMP version parallelizes the symbol loop.

A simplified representation is:

```
#pragma omp parallel for reduction(+:trades, position, pnl)
for (int s = 0; s < symbol_count; s++) {
    simulate_symbol(&symbols[s]);
}
```

Static scheduling is appropriate because the generated symbols contain similar numbers of events.

3.6.4 MPI Version

The MPI implementation distributes symbols among independent MPI processes or ranks.

A rank processes symbols according to:

$$s = \text{rank}, \text{rank} + \text{number of ranks}, \ldots$$

After local processing, values are combined using:

`MPI_Reduce()`

The total MPI runtime should be determined by the slowest participating rank because the entire job is complete only when all required ranks have completed their work.

4. System Workflow

The complete workflow of the project is shown in Figure 1.

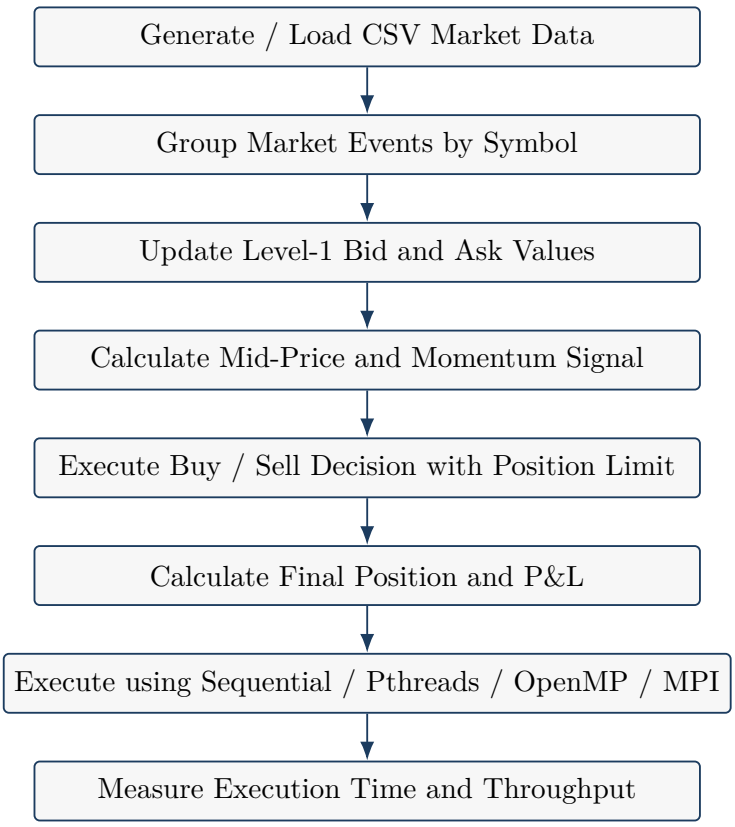


Figure 1: Workflow of the Mini HFT + HPC system

5. Implementation

The project is implemented primarily in the C programming language.

The important source files are summarized in Table 1.

Table 1: Major source files in the project

File	Purpose
`generate_data.c`	Generates deterministic synthetic market quote data.
`mini_hft.c`	Loads CSV data, updates the Level-1 order book, executes the momentum strategy and calculates P&L.
`sequential.c`	Runs the complete simulation sequentially.
`pthreads_version.c`	Implements explicit POSIX-thread based parallelism.
`openmp_version.c`	Implements shared-memory parallelism using OpenMP.
`mpi_version.c`	Implements process-based parallel execution using MPI.
`mini_hft.h`	Contains shared structures and function declarations.

6. Performance Metrics

The main HPC performance measurements used in the project are execution time and event-processing throughput.

6.1 Execution Time

Execution time is measured as:

$$T = T_{\text{end}} - T_{\text{start}}$$

A monotonic clock should be used so that changes in the system clock do not affect the measurement.

6.2 Throughput

Throughput represents the number of market events processed in one second.

$$\text{Throughput} = \frac{\text{Total Number of Events}}{\text{Execution Time}}$$

The unit is:

$$\text{events per second}$$

6.3 Speedup

When repeated timing experiments are available, speedup can be calculated as:

$$S_p = \frac{T_1}{T_p}$$

where:

- T_1 is the sequential execution time.
- T_p is the execution time with p parallel workers.

A speedup greater than one indicates that the parallel version completed the workload faster than the sequential implementation.

7. Validation and Current Results

The supplied test dataset contains four symbols and 10,000 events for each symbol.

Therefore:

$$4 \times 10,000 = 40,000$$

market events are processed.

The Sequential, Pthreads and OpenMP implementations produced identical financial results for the included validation workload. :chatgpt-content-referenceindex="1"

The observed values are shown in Table 2.

Table 2: Validated output for the test workload

Metric	**Observed Value**
Number of symbols	4
Events per symbol	10,000
Total market events	40,000
Total trades	27,999
Final combined position	-1
Final combined P&L	-207.625000

The identical trading results show that the tested parallel implementations preserve the same core simulation behavior.

Exact runtime values should not be treated as universal results because performance depends on:

- CPU architecture,
- number of cores,
- operating-system scheduling,
- compiler optimization,
- workload size, and
- thread or process configuration.

The MPI implementation is included in the project source code. However, the current package does not claim validated MPI runtime results because an MPI compiler/runtime was unavailable in the validation environment.

8. Limitations

The current mid-semester implementation has several limitations.

- The system uses synthetic market quote data rather than live exchange data.
- The Level-1 order book stores only the current best bid and ask rather than multiple price levels.

- The implementation is not a complete exchange matching engine.
- The momentum strategy is intentionally simple.
- Parallelism is mainly achieved across independent financial symbols.
- Small workloads may not show significant parallel speedup because thread and process-management overhead can dominate the actual computation.
- MPI performance still needs to be measured using a system with an installed MPI runtime.

9. Conclusion

The Mini HFT + HPC project demonstrates the application of High Performance Computing concepts to a simplified financial-market simulation.

The project processes deterministic synthetic quote data, maintains a Level-1 order book, applies a basic momentum trading strategy and calculates final trading P&L.

Four computing approaches are considered: Sequential C, Pthreads, OpenMP and MPI.

An important design feature is that all execution backends use the same core `simulate_symbol()` function. Therefore, changes in output can be separated from changes in the parallel execution method.

For the mid-semester stage, the project provides a clear and manageable demonstration of shared-memory multithreading, explicit threads and process-based parallelism using a practical event-processing application.

10. Future Scope

The project can be extended in several directions:

- Increase the number of financial symbols and market events.
- Perform multiple timing runs and calculate average execution time.
- Calculate speedup and parallel efficiency.
- Execute and benchmark the MPI version on a suitable MPI environment.
- Implement a multi-level limit order book.

- Add additional trading strategies.
- Compare dynamic and static workload scheduling.
- Explore hybrid MPI + OpenMP execution.
- Extend the project toward CUDA-based GPU acceleration.

References

[1] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," *AFIPS Spring Joint Computer Conference*, 1967.

[2] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*.

[3] MPI Forum, *MPI: A Message-Passing Interface Standard*.

[4] IEEE, *POSIX Threads Standard*.

[5] R. Cont and A. de Larrard, "Price Dynamics in a Markovian Limit Order Book," *SIAM Journal on Financial Mathematics*, vol. 4, no. 1, 2013.